

Chapter 7: Plate Recognition with AI

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
ALPR	Automatic License Plate Recognition
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
CV	Computer Vision
DHCP	Dynamic Host Configuration Protocol
DNN	Deep Neural Network
ECU	Electronic Control Unit
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
IoT	Internet of Things
JSON	JavaScript Object Notation
JPEG	Joint Photographic Experts Group
JWT	JSON Web Token
LPR	License Plate Recognition

Abbreviation	Full Term
MCU	Microcontroller Unit
ML	Machine Learning
mAP	Mean Average Precision
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PNG	Portable Network Graphics
REST	Representational State Transfer
RGB	Red, Green, Blue
ROI	Region of Interest
SQL	Structured Query Language
TTL	Time To Live
UI	User Interface
UART	Universal Asynchronous Receiver-Transmitter
URL	Uniform Resource Locator
WebSocket	Web Socket Protocol (RFC 6455)
Wi-Fi	Wireless Fidelity (IEEE 802.11)
YOLO	You Only Look Once

7.1 Introduction

In the operational context of a modern bus garage, the efficient and secure management of vehicle entry and exit is a critical logistical requirement. Traditionally, gate access control relied on manual inspection by security personnel — a process that is inherently slow, prone to human error, dependent on shift coverage, and incapable of producing a reliable audit trail. As fleet sizes grow and operational

throughput demands increase, manual gate management becomes a bottleneck that degrades both security and operational efficiency.

The **Automatic License Plate Recognition (ALPR)** subsystem of the Smart Bus Garage Management System directly addresses this operational challenge by automating the entire gate authorization workflow through the integration of Artificial Intelligence and computer vision. The subsystem captures images of vehicles approaching the garage entry and exit gates, detects and reads the license plate using a deep learning-based detection model combined with Optical Character Recognition, and transmits the resulting plate string and confidence score to the central FastAPI backend for authorization against the registered fleet database. The gate decision — grant or deny — is made programmatically within milliseconds, without any human intervention in the recognition or authorization loop.

The architectural approach adopted in this system is specifically that of **edge AI**: the computationally intensive inference tasks (plate detection and character recognition) are executed locally on the camera node itself, close to the physical data source, rather than being streamed as raw video to a remote server for cloud-based processing. This design choice is motivated by three primary engineering considerations. First, it eliminates the latency and bandwidth overhead of transmitting high-resolution video frames over the network continuously. Second, it reduces the processing burden on the central backend server, which is also responsible for serving REST API requests, managing WebSocket connections, and running scheduled background jobs. Third, it improves the system's resilience to network interruptions: the camera node can continue capturing and evaluating plates even during brief backend unavailability, queuing results for transmission when connectivity is restored.

The backend integration layer, implemented as a FastAPI webhook endpoint (`POST /api/v1/hardware/anpr`), receives the pre-computed plate recognition results, applies a confidence threshold filter, performs vehicle authorization against the PostgreSQL database via SQLAlchemy, persists a complete audit log of every gate event in the `gate_logs` table, and broadcasts real-time access notifications to connected Manager and Admin clients via WebSocket. This chapter documents the complete technical architecture of this subsystem, from the image capture event at the physical camera through the AI inference pipeline on the edge device, through the authorization logic in the backend, to the gate actuation response and real-time operator notification.

7.2 System Architecture: Edge AI Design

7.2.1 Rationale for Edge Computing

The ALPR subsystem follows a distributed **edge computing** architecture [1], in which the AI inference pipeline is co-located with the image source rather than centralized in the backend server. This pattern is increasingly standard in modern IoT and computer vision deployments [2] and is particularly appropriate for the garage gate control use case for the following reasons:

- **Latency:** Gate access decisions are time-critical. A vehicle approaches a gate at low speed, and the gate actuator must respond before the vehicle reaches the barrier. By processing the image locally on the camera node, end-to-end latency from image capture to gate decision is reduced to the sum of local inference time (typically 50–200 ms for an optimized YOLOv8 [3] model on edge hardware) plus HTTP transmission time, rather than also including the round-trip for image upload to a remote server.
- **Bandwidth:** Transmitting raw camera frames (each 1–5 MB as JPEG at typical surveillance resolutions) at 10–30 FPS to a central server would require sustained network throughput of 10–150 Mbps per camera. Transmitting only the inference result — a JSON payload of under 200 bytes — reduces per-camera bandwidth consumption by more than three orders of magnitude.
- **Backend decoupling:** The FastAPI backend is designed as a stateless REST API server, not a video processing engine. Delegating inference to the edge preserves the backend's architectural clarity and keeps its resource requirements proportional to the number of API requests, not to camera frame rates.
- **Resilience:** The edge camera can buffer recognition results locally and retry transmission if the backend is temporarily unreachable, ensuring no gate events are lost during brief network interruptions.

7.2.2 Component Roles

The ALPR subsystem comprises four distinct component layers, each with a well-defined role. Critically, the edge layer is split into two physically separate devices — a capable **Edge AI PC** (local laptop or desktop) that runs the computationally intensive AI pipeline, and a lightweight **ESP32 Gate Actuator** that handles only physical gate control via GPIO:

Component	Location	Role
Edge AI PC (Local Laptop/Desktop)	Physical / Edge Layer	Camera feed capture (USB/IP camera via OpenCV), image preprocessing, plate detection (YOLOv8 [3]), OCR (EasyOCR), confidence

Component	Location	Role
		aggregation, HTTP POST to backend, and local HTTP trigger to ESP32 upon GRANTED response
ESP32 Gate Actuator	Physical / Edge Layer	Lightweight HTTP server on the LAN; receives actuation commands from the Edge AI PC; drives GPIO pins to open/close the boom barrier servo; provides no AI or authorization logic
FastAPI Backend (/app/api/hardware.py)	Backend Processing Layer	Webhook ingestion, confidence threshold validation, database authorization query, GateLog persistence, WebSocket broadcast
React Web Client	Presentation Layer	Real-time display of gate events via WebSocket; historical audit log via REST API

Important architectural note: The ESP32 microcontroller lacks the computational resources (RAM, CPU, GPU) to execute deep learning models such as YOLOv8 or EasyOCR. Therefore, all AI inference is performed on the Edge AI PC, which has a general-purpose CPU (and optionally a GPU). The ESP32 serves exclusively as a remote-controlled gate actuator on the same local network.

7.2.3 Overall Data Flow

The end-to-end data flow of a single gate event, from physical vehicle arrival to operator notification, follows this pathway:

Camera captures frame → Edge AI pipeline extracts plate string + confidence → HTTP POST to /api/v1/hardware/anpr → Confidence threshold filter → Database vehicle lookup → Authorization decision → GateLog record written → WebSocket broadcast → Manager browser receives alert

This flow is illustrated in full detail in the sequence diagram in Section 7.7.2.

7.3 Image Acquisition

7.3.1 Gate Camera Hardware

Gate cameras are deployed at the garage entry, exit, or combined entry/exit points. Each camera's deployment configuration is registered in the PostgreSQL database using the `GateCamera` model (`__tablename__ = "gate_cameras"`), which stores the following attributes:

```
class GateCamera(Base):
    __tablename__ = "gate_cameras"

    id : Integer, primary_key
    location_name: String(255) # e.g., "Main Entry Gate North"
    ip_address : String(50), unique # Camera's static IP on the local network
    gate_type : String(50) # "ENTRY" | "EXIT" | "BOTH"
    is_active : Boolean, default=True
    created_at : DateTime(timezone=True)
    updated_at : DateTime(timezone=True)
```

This registry enables the backend to correlate incoming ANPR events with their physical gate of origin (via the `gate_id` field in the ANPR webhook payload) and to flag events from decommissioned or inactive cameras. The `gate_type` attribute distinguishes entry-only, exit-only, and bidirectional cameras, allowing the system to apply different authorization logic for each gate position in future extensions.

7.3.2 Image Capture Trigger

Gate cameras operate in continuous monitoring mode, analyzing the video stream frame-by-frame at the edge node. Two triggering strategies are common in production ALPR deployments [4]:

- **Motion-triggered capture:** The camera applies a frame-differencing algorithm to detect movement within the gate approach zone (a defined region of interest). When motion is detected, the full inference pipeline is activated for the incoming frames until the vehicle passes through or a timeout elapses. This strategy conserves edge processing resources during idle periods.
- **Periodic frame sampling:** The edge node samples frames at a fixed interval (e.g., 5–10 FPS) and passes each frame through a lightweight vehicle presence detector before invoking the full ALPR pipeline. This approach is simpler to implement but less computationally efficient.

In either case, once a candidate frame containing a vehicle is identified, the full ALPR processing pipeline described in Section 7.4 is invoked on that frame.

7.3.3 Camera-to-Edge Processor Interface

When the camera node and the inference processor are separate physical units (e.g., an IP camera feeding frames to a co-located single-board computer such as a Raspberry Pi or NVIDIA Jetson), the video stream is typically delivered over the local network using the RTSP (Real-Time Streaming Protocol) or directly via USB/CSI interface if integrated. The edge processor reads frames using OpenCV's [5] VideoCapture API:

```
import cv2

cap = cv2.VideoCapture("rtsp://camera_ip:554/stream") # Network camera
# or
cap = cv2.VideoCapture(0) # Integrated USB/CSI camera

ret, frame = cap.read() # Capture a single frame as a NumPy array (H x W x 3, BGR)
```

The captured frame is a NumPy [6] array of shape (H, W, 3) in BGR color order, which serves as the input to the preprocessing stage.

7.4 The ALPR Processing Pipeline

The complete AI processing pipeline transforms a raw camera frame into a validated, normalized plate number string accompanied by a confidence score. This pipeline executes entirely on the edge device before any data is transmitted to the backend. The pipeline consists of four sequential stages: image preprocessing, license plate region detection, character recognition, and confidence aggregation.

7.4.1 AI Processing Pipeline Diagram

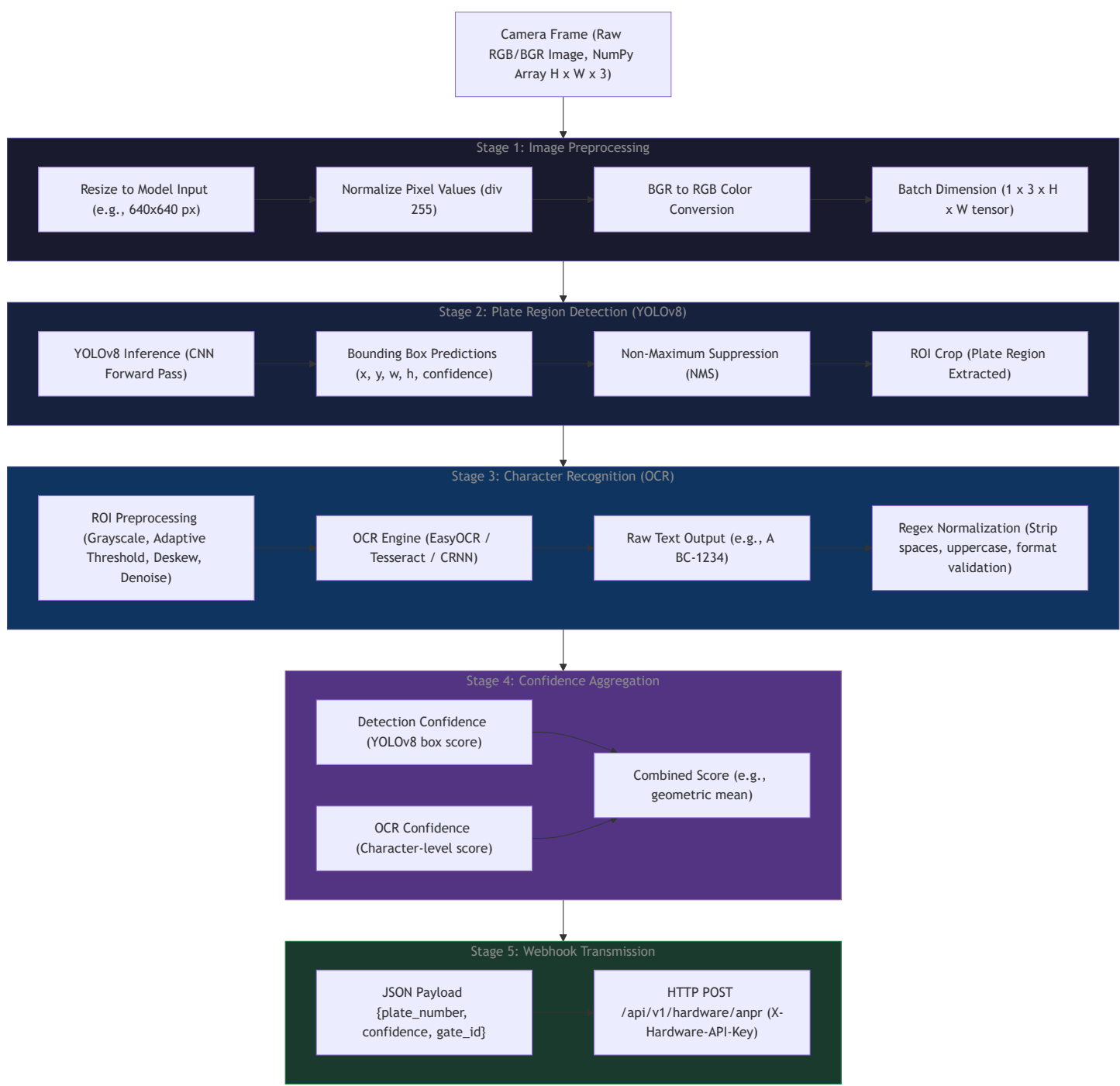


Figure 7.1 — The five-stage ALPR processing pipeline executing on the Edge AI PC, from raw camera frame capture through deep learning plate detection, OCR character extraction, confidence scoring, and HTTP webhook transmission to the FastAPI backend.

7.4.2 Stage 1: Image Preprocessing

Before the raw camera frame is passed to the deep learning detector, a series of preprocessing operations are applied to normalize the input and improve detection reliability [5]:

Resizing: YOLOv8 [3] models are trained on fixed-resolution inputs (typically 640 x 640 pixels for the standard models). The captured frame — which may be 1080p (1920 x 1080) or higher — is resized to the model's expected input dimensions using bicubic interpolation. Maintaining aspect ratio through letterboxing (padding with gray pixels) prevents geometric distortion of the plate characters.

Color space normalization: OpenCV [5] reads images in BGR channel order by default. The frame is converted to RGB using `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` before being passed to the YOLOv8 [3] model, which expects RGB input. Pixel values are normalized from the integer range `[0, 255]` to the floating-point range `[0.0, 1.0]` by dividing by 255.

Tensor conversion: The NumPy [6] array of shape `(H, W, 3)` is transposed to channel-first format `(3, H, W)` and an additional batch dimension is prepended to yield the final input tensor of shape `(1, 3, H, W)`, compatible with the PyTorch [7] inference runtime used by YOLOv8.

Contrast enhancement (optional): In low-light conditions — such as nighttime or covered garage approaches — Contrast Limited Adaptive Histogram Equalization (CLAHE) [5] is applied to the luminance channel of the LAB color space representation to enhance the visibility of plate characters before detection.

7.4.3 Stage 2: License Plate Region Detection (YOLOv8)

License plate detection is performed by a **YOLOv8** [3] (You Only Look Once, version 8) object detection model from Ultralytics, fine-tuned on a dataset of vehicle license plates. YOLOv8 belongs to the family of single-stage object detectors that, unlike two-stage detectors (e.g., Faster R-CNN), perform bounding box regression and class prediction in a single forward pass through the network, yielding significantly lower inference latency — a critical property for real-time gate control.

Model Architecture: YOLOv8's backbone is a modified CSPDarknet CNN [3] that extracts hierarchical feature representations at multiple scales. The neck employs a Path Aggregation Network (PANet) to merge features from different backbone levels, enabling detection of plates at varying distances and sizes. The head outputs a grid of predictions, each predicting bounding box coordinates `(x_center, y_center, width, height)`, an objectness score, and a class probability for the license plate class.

Inference: The preprocessed tensor is passed through the YOLOv8 model:

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n_plate.pt") # Nano model fine-tuned for plate detection
results = model(frame, conf=0.5, iou=0.45) # Confidence and IoU thresholds
```

Non-Maximum Suppression (NMS): YOLOv8 applies NMS internally to eliminate overlapping bounding box predictions for the same physical plate, retaining only the highest-confidence detection. The IoU threshold of 0.45 controls the aggressiveness of suppression — boxes whose Intersection over Union with the top prediction exceeds 0.45 are discarded.

ROI Extraction: The highest-confidence bounding box is used to crop the plate Region of Interest (ROI) from the original full-resolution frame using NumPy [6] array slicing:

```
x1, y1, x2, y2 = map(int, results[0].boxes.xyxy[0])
plate_roi = frame[y1:y2, x1:x2] # Cropped plate region
detection_confidence = float(results[0].boxes.conf[0])
```

This cropped `plate_roi` image is passed to the OCR stage for character extraction.

7.4.4 Stage 3: Optical Character Recognition (OCR)

The cropped plate ROI undergoes a second round of preprocessing specifically optimized for OCR accuracy [8] before text extraction is performed:

Grayscale Conversion: The color ROI is converted to a single-channel grayscale image using `cv2.cvtColor(plate_roi, cv2.COLOR_RGB2GRAY)`. Color information is not required for character recognition and the single-channel representation reduces the input dimensionality for the OCR model.

Adaptive Thresholding: `cv2.adaptiveThreshold()` converts the grayscale image to a binary (black/white) image, isolating dark characters against the plate's light background regardless of uneven illumination across the plate surface. Adaptive thresholding adjusts the threshold value locally for each pixel based on its neighborhood, making it robust to shadows and non-uniform lighting [5].

Deskewing: Plate images may be slightly rotated due to camera angle or vehicle inclination. A deskew correction step computes the minimum area bounding rectangle of the plate region and applies an affine rotation transform to align it horizontally, improving OCR character segmentation.

Denoising: `cv2.fastNlMeansDenoising()` removes salt-and-pepper noise artifacts that can cause the OCR engine to misclassify characters.

The preprocessed binary plate image is then passed to the OCR engine. The system is compatible with two OCR backends:

- **EasyOCR** [9]: A deep learning-based OCR library that uses a CRNN (Convolutional Recurrent Neural Network) architecture to perform end-to-end text recognition. EasyOCR [9] provides per-

word confidence scores and supports Arabic and Latin scripts without separate language configuration, making it suitable for the Egyptian license plate format used in this deployment.

- **Tesseract OCR** [8] (with `pytesseract` Python wrapper): A traditional OCR engine that uses an LSTM-based neural network for character sequence recognition. Tesseract [8] can be configured with a custom character whitelist (digits and letters only:

`config="--psm 7 -c tesseract_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789" )` to prevent the engine from predicting invalid symbols that cannot appear on a license plate.

```
import easyocr

reader = easyocr.Reader(['en'], gpu=False)
ocr_results = reader.readtext(preprocessed_plate, detail=1)
# Returns: [(bounding_box, text, confidence), ...]
plate_text = ocr_results[0][1]          # Extracted text string
ocr_confidence = ocr_results[0][2]      # Per-prediction confidence (0.0-1.0)
```

Post-processing and Normalization: The raw OCR output string undergoes regular expression normalization to produce a clean, canonical plate number:

```
import re

normalized = re.sub(r'^A-Z0-9+', '', plate_text.upper().strip())
# e.g., "A BC - 1234" -> "ABC1234"
```

This normalization removes spaces, hyphens, and any non-alphanumeric characters introduced by OCR misdetection, producing a string that matches the format stored in the `vehicles.plate_number` database column.

7.4.5 Stage 4: Confidence Score Aggregation

The final confidence score submitted to the backend webhook represents a combined measure of both the plate detection quality (from YOLOv8 [3]) and the character recognition quality (from the OCR engine [9]). A geometric mean combination is commonly used [4] to produce a balanced score that is sensitive to low quality in either stage:

```
combined_confidence = sqrt(detection_confidence x ocr_confidence)
```

For example, a detection confidence of 0.94 and an OCR confidence of 0.91 yield a combined score of $\sqrt{0.94 \times 0.91} = 0.924$. This combined score is the `confidence` value transmitted in the JSON

webhook payload to `POST /api/v1/hardware/anpr` .

7.5 Backend Integration and Authorization Logic

7.5.1 The ANPR Webhook Endpoint

Once the edge device has completed its ALPR pipeline and produced a normalized plate string and confidence score, it transmits the result to the FastAPI backend via an HTTP POST request to `POST /api/v1/hardware/anpr` , authenticated with the hardware API key:

```
POST /api/v1/hardware/anpr
X-Hardware-API-Key: <cryptographic-secret>
Content-Type: application/json
```

```
{
  "plate_number": "ABC1234",
  "confidence": 0.924,
  "gate_id": "GATE_ENTRY_01"
}
```

This payload is validated against the `ANPRData` Pydantic schema defined in `/app/schemas/schemas.py` :

```
class ANPRData(BaseModel):
    plate_number: str
    confidence: float
    gate_id: str
```

Pydantic enforces type correctness and rejects malformed payloads (e.g., non-numeric confidence values, missing fields) with an automatic `HTTP 422 Unprocessable Entity` response, before the handler logic is reached.

7.5.2 Confidence Threshold Validation

The `ingest_anpr()` handler, implemented in `/app/api/hardware.py` , first applies a confidence threshold check as the primary quality gate:

```

if data.confidence < 0.85:
    # Log as IGNORED – recognition quality insufficient for authorization
    logger.info(
        f"AUDIT | Gate {data.gate_id} DENIED | "
        f"Plate: {redacted_plate} | "
        f"Confidence: {data.confidence:.2f} (Below 0.85)"
    )
    event_type = "IGNORED"

```

The threshold of **0.85** (85%) is configured as a system constant. Readings with a confidence below this threshold are classified as `IGNORED`: they are logged to the `gate_logs` table for audit purposes but do not trigger a database vehicle lookup, and no WebSocket notification is broadcast. This prevents low-quality OCR outputs — produced under poor lighting, partial occlusion, or motion blur — from generating false authorization decisions.

Crucially, the handler applies a **plate number redaction** function before logging:

```

def redact_plate(plate: str) -> str:
    if not plate or len(plate) < 4:
        return "****"
    return f"{plate[:2]}***{plate[-2:]}"

```

This ensures that full plate numbers are never exposed in application logs, protecting vehicle owner privacy in compliance with data minimization principles. The full, unredacted plate number is stored only in the database's `gate_logs` table, which is access-controlled.

7.5.3 Vehicle Database Authorization

For readings with `confidence >= 0.85`, the handler performs an authorized vehicle lookup against the `vehicles` table using SQLAlchemy's async ORM:

```

vehicle = await db.scalar(
    select(Vehicle).where(
        Vehicle.plate_number == data.plate_number,
        Vehicle.status.in_(
            VehicleStatus.EN_ROUTE,
            VehicleStatus.ASSIGNED,
            VehicleStatus.FREE
        )
    )
)

```

This query implements the system's authorization policy: a vehicle is granted access if and only if:

1. Its plate number exists in the registered fleet database, **AND**
2. Its current operational status is one of `EN_ROUTE` , `ASSIGNED` , or `FREE` .

Vehicles in `MAINTENANCE` or `OUT_OF_SERVICE` status are denied access even if their plate number is registered, preventing decommissioned or unserviceable buses from entering or leaving the garage. This status-aware authorization is a key security feature that ensures the access control system is coupled to the fleet's real-time operational state.

The authorization decision is determined as follows:

```

if vehicle:
    event_type = "GRANTED"
else:
    event_type = "DENIED"
    # No matching active vehicle found for this plate

```

7.5.4 Gate Event Persistence

Regardless of the authorization outcome, every ANPR event is persisted as a `GateLog` record in the PostgreSQL database via SQLAlchemy:

```

gate_log = GateLog(
    gate_id=data.gate_id,
    plate_number=data.plate_number,
    confidence=data.confidence,
    event=event_type,          # "GRANTED" | "DENIED" | "IGNORED"
    vehicle_id=vehicle.id if vehicle else None,
)
db.add(gate_log)
await db.commit()

```

The GateLog model schema is:

```

class GateLog(Base):
    __tablename__ = "gate_logs"

    id          : Integer, primary_key
    gate_id     : String(50)           # Physical gate identifier
    plate_number: String(50)           # Full plate string (unredacted, access-controlled)
    confidence  : Float                # Combined AI confidence score
    event       : String(50)           # "GRANTED" | "DENIED" | "IGNORED"
    vehicle_id  : ForeignKey("vehicles.id"), nullable
    created_at  : DateTime(timezone=True)

    vehicle     : relationship -> Vehicle

```

This persistent audit trail records every vehicle interaction with the garage gates, enabling post-incident investigation, compliance reporting, and statistical analysis of gate utilization patterns.

7.5.5 Gate Actuation Response — Two-Device Communication Bridge

The physical gate actuation involves a **two-device communication bridge** between the Edge AI PC and the ESP32 Gate Actuator. This separation is architecturally necessary because the AI inference pipeline (YOLOv8 + EasyOCR) requires computational resources that exceed the ESP32's capabilities, while the physical GPIO pins that drive the servo motors reside on the ESP32 microcontroller.

Step 1 — Backend responds to Edge AI PC: The HTTP response body returned to the Edge AI PC by `ingest_anpr()` carries the authorization decision:

```

# For GRANTED:
return {
    "type": "gate_auth",
    "event": "GATE_AUTH_GRANTED",
    "gate_id": data.gate_id,
    "plate_number": data.plate_number,
    "vehicle_id": vehicle.id,
    "message": f"Access granted to {data.plate_number}"
}

# For DENIED:
return {
    "type": "gate_auth",
    "event": "UNAUTHORIZED_VEHICLE",
    "gate_id": data.gate_id,
    "plate_number": data.plate_number,
    "message": f"Unauthorized access attempt by {data.plate_number}"
}

# For IGNORED:
return {"status": "ignored", "reason": "low confidence"}

```

Step 2 — Edge AI PC signals ESP32 Gate Actuator: The Edge AI PC (Python script running on a local laptop) reads the `event` field from the backend's HTTP response. If `event == "GATE_AUTH_GRANTED"`, the PC sends a lightweight local HTTP request to the ESP32 on the same WiFi LAN:

```

# Edge AI PC – post-authorization gate trigger
import requests

response_data = backend_response.json()
if response_data.get("event") == "GATE_AUTH_GRANTED":
    gate_id = response_data["gate_id"]
    esp32_url = f"http://192.168.1.8/open?gate={gate_id}" # ESP32 local server
    requests.get(esp32_url, timeout=5)

```

Step 3 — ESP32 actuates the physical barrier: The ESP32 (`sketch_apr7a.ino`) runs a lightweight HTTP server on the LAN. Upon receiving the `/open` request from the Edge AI PC, it asserts the appropriate GPIO output pin HIGH via its LEDC PWM peripheral, activating the servo motor that drives the boom barrier open. After a configurable delay (4,000 ms as defined by `GATE_OPEN_MS`), the GPIO pin is returned to the closed position, allowing the barrier to close automatically:

```
// ESP32 Gate Actuator — sketch_apr7a.ino
void setGateIn(bool open) {
    ledcWrite(SERVO_IN, angleToDuty(open ? 90 : 0));
}
// Gate held open for GATE_OPEN_MS (4000ms), then auto-closes
```

If the backend response event is `DENIED` or `IGNORED`, the Edge AI PC does **not** send any request to the ESP32, and the barrier remains closed.

This two-device architecture is clean and well-separated: the Edge AI PC handles all intelligence (AI inference and response interpretation), while the ESP32 is a simple, reliable actuator that responds to local network commands. Neither device performs authorization logic — that responsibility belongs exclusively to the FastAPI backend, which has access to the authoritative fleet database.

7.5.6 Real-Time WebSocket Notification

Concurrently with returning the HTTP response, the handler broadcasts the gate event to all connected Manager and Admin role clients via the `WebSocket ConnectionManager`:

```
if event_type == "GRANTED":
    alert_payload = {
        "type": "gate_auth",
        "event": "GATE_AUTH_GRANTED",
        "gate_id": data.gate_id,
        "plate_number": data.plate_number,
        "vehicle_id": vehicle.id,
        "message": f"Access granted to {data.plate_number}"
    }
elif event_type == "DENIED":
    alert_payload = {
        "type": "gate_auth",
        "event": "UNAUTHORIZED_VEHICLE",
        "gate_id": data.gate_id,
        "plate_number": data.plate_number,
        "message": f"Unauthorized access attempt by {data.plate_number}"
    }

await manager.broadcast_to_role("MANAGER", alert_payload)
await manager.broadcast_to_role("ADMIN", alert_payload)
```

The `broadcast_to_role()` method publishes the alert to the Redis Pub/Sub channel, which the WebSocket supervisor coroutine delivers to all active Manager and Admin WebSocket connections. On the React frontend, the `websocketContext` receives the `gate_auth` message, calls `alertStore.addAlert()` to append the event to the real-time notification store, and the manager's notification bell counter increments. The full gate event — timestamp, plate, gate ID, and decision — is immediately visible in the Manager's notification panel without requiring a page refresh or manual query.

7.6 Data Persistence and Analytics

7.6.1 GateLog as the Primary Audit Table

The `gate_logs` table serves as the primary source of truth for all historical gate access data. Every ANPR event — including ignored low-confidence reads — is permanently stored with a UTC timestamp. This comprehensive persistence policy ensures that:

- Security incidents can be investigated by querying the full history of access attempts for any plate number or gate.
- Pattern analysis can identify unusual access times, repeated denied attempts from the same plate, or abnormally high volumes of low-confidence reads (which may indicate camera misalignment or vandalism).
- Compliance reporting can demonstrate that all vehicle movements in and out of the garage are recorded.

The Administrator role can retrieve the complete gate log history through the admin interface, with filtering by date range, gate ID, event type, and plate number. The audit log viewer page (`/admin/AuditLogs.jsx`) provides a paginated, searchable display of these records.

7.6.2 Vehicle Status Coupling

A key design property of the authorization query (Section 7.5.3) is that it does not merely check whether a plate is registered — it also checks the vehicle's current operational `status`. This means the gate authorization system is **dynamically coupled** to the fleet management system's real-time state:

- When a maintenance request for a vehicle is approved and its status transitions to `MAINTENANCE`, that vehicle is immediately denied gate access, even if it was previously authorized.

- When a vehicle is returned to `FREE` status after maintenance completion, it is automatically re-authorized without any manual configuration change.
- Vehicles that are `OUT_OF_SERVICE` (e.g., due to a critical IoT sensor alert as described in Chapter 6) are automatically excluded from gate access.

This dynamic coupling, implemented through SQLAlchemy's live ORM queries rather than a static whitelist, ensures that the access control policy always reflects the current operational reality of the fleet without requiring a separate synchronization mechanism.

7.7 System Workflow Sequence Diagrams

7.7.1 ANPR System Sequence Diagram

The following diagram traces the complete communication and logic sequence from the physical moment a bus approaches the garage gate through to the gate opening and the manager receiving a real-time notification. Note the separation of the **Edge AI PC** (which runs the AI pipeline) and the **ESP32 Actuator** (which controls the physical gate).

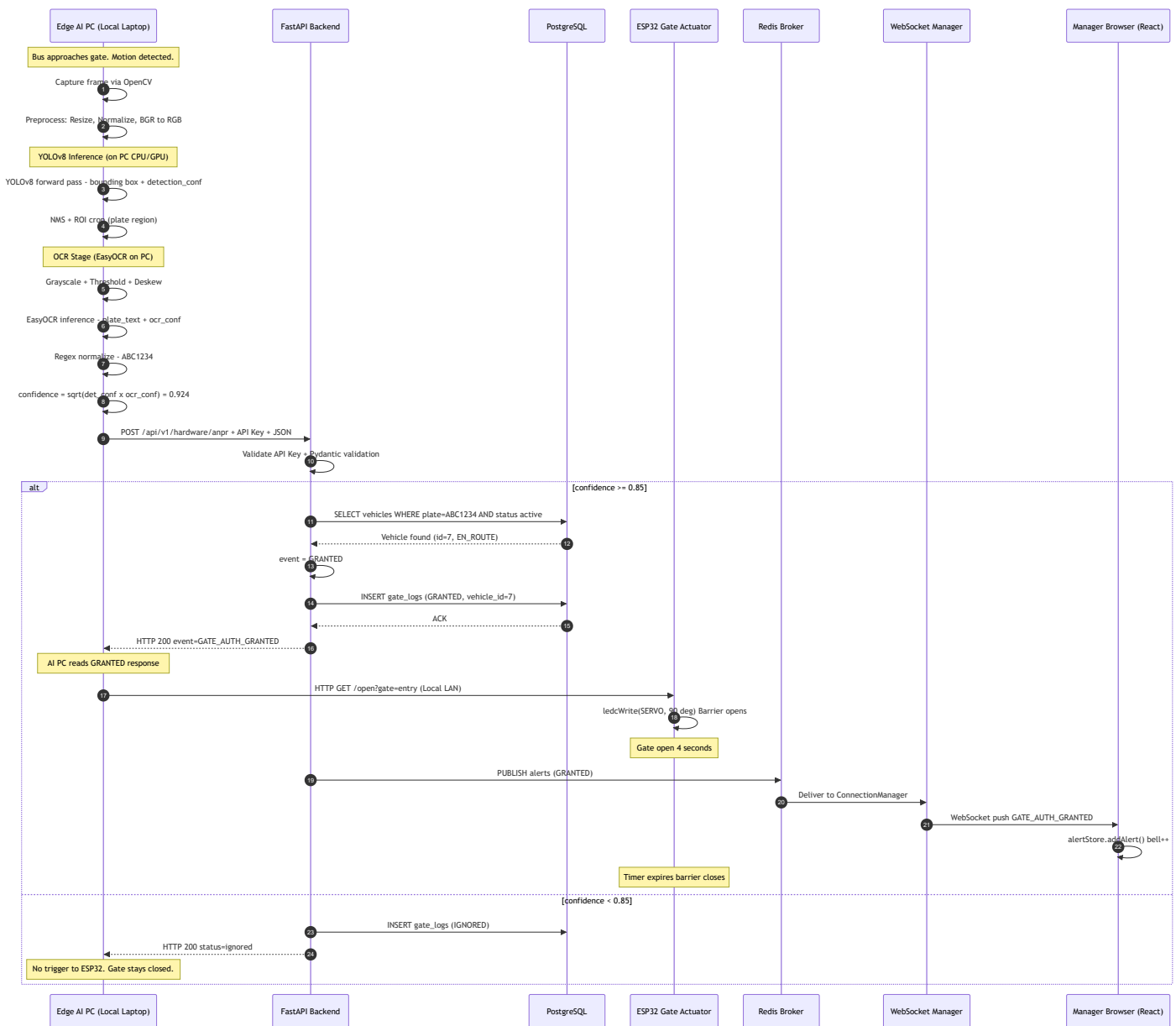


Figure 7.2 — Complete ANPR system sequence diagram showing the Edge AI PC running inference, posting to the backend, receiving the authorization, and signalling the ESP32 Gate Actuator via local HTTP to open the barrier.

7.7.2 Denied Access Scenario

For completeness, the following diagram illustrates the denied access workflow — when a plate is recognized with sufficient confidence but the vehicle is not found in the authorized fleet database.

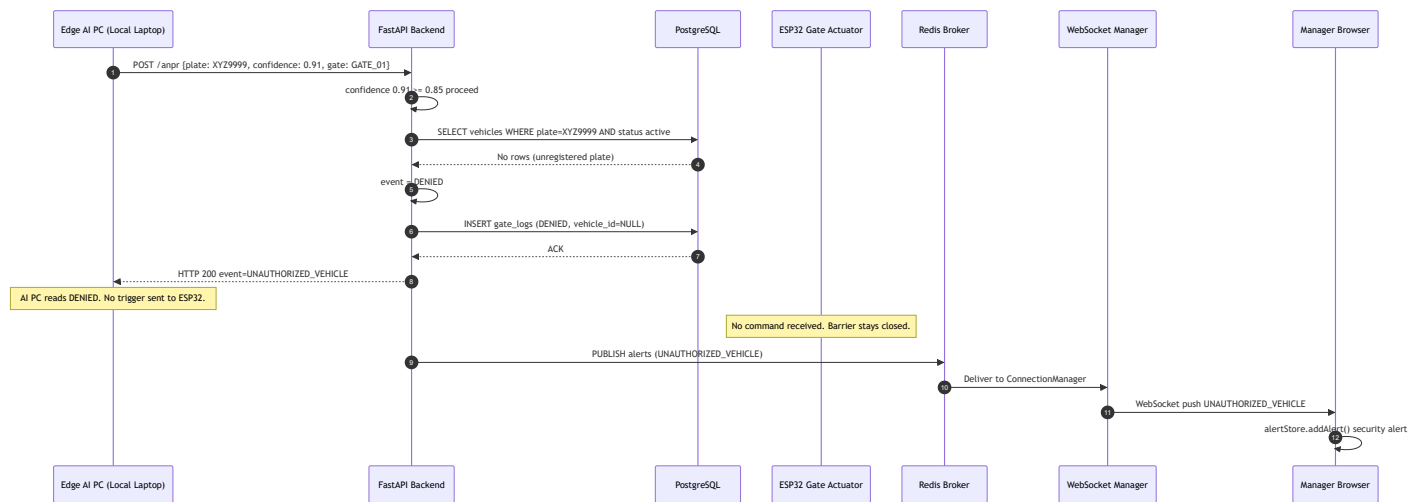


Figure 7.3 — Denied access sequence: an unregistered vehicle's plate is recognized with high confidence but fails the database authorization check, triggering a security alert to the manager without opening the gate.

7.8 Security Considerations in the ALPR Subsystem

7.8.1 Plate Number Privacy

As documented in Section 7.5.2, the `redact_plate()` function masks the plate number in all application log output to `xx***yy` format. Full plate numbers are stored exclusively in the database's `gate_logs` table, which is accessible only through the authenticated admin interface to users with the `ADMIN` role. This implements a data minimization principle consistent with data protection regulations, ensuring that plate numbers — which are considered personally identifiable information in many jurisdictions — are not exposed in server log files that may have broader access.

7.8.2 Confidence Threshold as a Security Gate

The 0.85 confidence threshold serves a dual purpose: it ensures recognition quality sufficient for reliable authorization decisions, and it acts as a security measure against spoofing attacks. A physical spoofing attempt using a printed or altered plate would typically produce a lower confidence score due to font inconsistency, reflective material differences, or dimensional distortion, increasing the probability that the attempt falls below the threshold and is classified as `IGNORED` rather than triggering an authorization lookup.

7.8.3 Status-Aware Authorization

As discussed in Section 7.6.2, the authorization query's `Vehicle.status.in_(...)` clause ensures that only operationally active vehicles can gain access. This prevents a scenario in which a registered but decommissioned vehicle — whose physical plate may have been transferred or cloned — could gain unauthorized access by exploiting a stale whitelist.

7.8.4 Hardware API Key Protection

The ANPR endpoint is protected by the pre-shared hardware API key transmitted in the `X-Hardware-API-Key` header. The backend uses `hmac.compare_digest()` for constant-time comparison to prevent timing side-channel attacks. In production, all communication between the camera node and backend is secured by TLS (HTTPS), ensuring the API key and plate number data are encrypted in transit.

7.9 Summary of the ALPR Subsystem

The following table provides a consolidated reference for all key parameters, thresholds, and implementation details of the ALPR subsystem documented in this chapter.

Parameter / Component	Value / Detail
AI Architecture	Edge computing — inference on local PC, not backend server or ESP32
Edge AI Device	Local laptop/desktop PC running Python (YOLOv8 + EasyOCR)
Gate Actuator Device	ESP32 MCU — receives open/close commands via local HTTP from the AI PC
Plate Detection Model	YOLOv8 (Ultralytics) [3], fine-tuned for license plates
OCR Engine	EasyOCR [9] (CRNN-based) / Tesseract [8] (LSTM-based)
Image Processing Library	OpenCV [5] (cv2)

Parameter / Component	Value / Detail
Numerical Computing	NumPy [6]
Preprocessing Steps	Resize, normalize, BGR to RGB, grayscale, adaptive threshold, deskew, denoise
Confidence Aggregation	Geometric mean of detection and OCR scores
Confidence Threshold	0.85 (configured in <code>/app/api/hardware.py</code>)
Below-threshold action	IGNORED — logged but no authorization lookup or gate action
Authorization DB Query	<code>SELECT WHERE plate_number = ? AND status IN (EN_ROUTE, ASSIGNED, FREE)</code>
Database Model	GateLog (<code>gate_logs</code> table)
Gate Camera Registry	GateCamera (<code>gate_cameras</code> table)
Grant Response	Backend to AI PC: <code>{"event": "GATE_AUTH_GRANTED"}</code> then AI PC to ESP32: <code>HTTP /open</code> then GPIO HIGH then Barrier opens
Deny Response	Backend to AI PC: <code>{"event": "UNAUTHORIZED_VEHICLE"}</code> then No signal to ESP32 then Barrier closed
Real-time Notification	WebSocket broadcast via Redis Pub/Sub to MANAGER + ADMIN roles
Plate Log Redaction	<code>xx***yy</code> format in application logs
Backend Endpoint	<code>POST /api/v1/hardware/anpr</code> (Rate limit: 60 req/min)
Backend File	<code>/app/api/hardware.py</code> — <code>ingest_anpr()</code> function

Parameter / Component	Value / Detail
Schema File	/app/schemas/schemas.py — ANPRData Pydantic model

References

[1] W. Shi *et al.*, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, Oct. 2016. doi: 10.1109/JIOT.2016.2579198.

[2] N. D. Lane and P. Warden, "The Deep (Learning) Transformation of Mobile and Embedded Computing," *IEEE Computer*, vol. 51, no. 5, pp. 12-16, May 2018. doi: 10.1109/MC.2018.2381129.

[3] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8 Documentation*, Ultralytics Inc., 2023. [Online]. Available: <https://docs.ultralytics.com>. [Accessed: Apr. 20, 2026].

[4] H. Li, P. Wang, and C. Shen, "Towards End-to-End Car License Plates Detection and Recognition with Deep Neural Networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 3, pp. 1126-1136, Mar. 2019. doi: 10.1109/TITS.2018.2847291.

[5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000. [Online]. Available: <https://opencv.org>. [Accessed: Apr. 20, 2026].

[6] C. R. Harris *et al.*, "Array Programming with NumPy," *Nature*, vol. 585, pp. 357-362, Sep. 2020. doi: 10.1038/s41586-020-2649-2.

[7] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Curran Associates, Inc., 2019, pp. 8024-8035. [Online]. Available: <https://pytorch.org/docs/stable/index.html>. [Accessed: Apr. 20, 2026].

[8] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR 2007)*, Curitiba, Brazil, Sep. 2007, pp. 629-633. doi: 10.1109/ICDAR.2007.4376991.

[9] J. Baek *et al.*, "What Is Wrong with Scene Text Recognition Model Comparisons? Dataset and Model Analysis," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Korea, Oct. 2019, pp. 4715-4723. doi: 10.1109/ICCV.2019.00481. [EasyOCR implementation: <https://github.com/JaidedAI/EasyOCR>. Accessed: Apr. 20, 2026].

- [10]** J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779-788. doi: 10.1109/CVPR.2016.91.
- [11]** S. Ramirez, *FastAPI Documentation*, Tiangolo, 2024. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: Apr. 20, 2026].
- [12]** Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.3, Espressif Systems Ltd., Shanghai, China, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. [Accessed: Apr. 20, 2026].
- [13]** M. Bayer, *SQLAlchemy 2.0 Documentation*, SQLAlchemy Project, 2024. [Online]. Available: <https://docs.sqlalchemy.org/en/20/>. [Accessed: Apr. 20, 2026].
- [14]** S. Colvin *et al.*, *Pydantic V2 Documentation*, Pydantic, 2024. [Online]. Available: <https://docs.pydantic.dev/latest/>. [Accessed: Apr. 20, 2026].
- [15]** Z. Zherzdev and A. Gruzdev, "EasyOCR: Ready-to-use OCR with 80+ Supported Languages," GitHub Repository, JaidedAI, 2020. [Online]. Available: <https://github.com/JaidedAI/EasyOCR>. [Accessed: Apr. 20, 2026].